Cat and Dog Breed Recognition with a Reject Class

Softwareentwicklungspraktikum: Computer Vision & Deep Learning

Doga Budakci Doga Erdag Eren Degirmenci Orhun Mahirogullari
{Doga.Budakci,Doga.Erdag,E.Degirmenci,Orhun.Mahirogullari}@campus.lmu.de

Abstract

This study investigates fine-grained cat and dog breed recognition with explicit rejection of non-target images. The task includes 20 target breeds and one reject class. A custom CNN (Convolutional Neural Network) was compared with ResNet18, EfficientNet-B0, and Swin Transformer Tiny under from-scratch and transfer-learning settings. The proposed pipeline combines data augmentation, YOLOv8n-based localization, confidence-threshold calibration, and weighted softmax ensembling. Grad-CAM was used to observe which image regions the model focused on. Performance was evaluated using accuracy, macro-F1, macro-precision, macro-recall, reject-class F1, false accepts, and false rejects. After evaluating multiple configurations for each model, ensembling was found to provide a substantial improvement. Therefore, the optimized from-scratch models and the optimized pretrained models were combined into separate ensembles, which were selected as the final candidate models. The from-scratch ensemble achieved 82.30% validation accuracy and macro-F1 of 83.84%. The pretrained ensemble produced the best overall results, reaching 95.48% validation accuracy and macro-F1 of 95.95%. Overall, ensemble-based transfer learning provided the strongest breed recognition and rejection performance.

1. Introduction

This project investigates the classification of 10 cat breeds and 10 dog breeds while rejecting non-target images. The approach learns discriminative breed features and remains robust to uncertain inputs. Several architectures were evaluated to address these requirements.

1.1. Motivation and Challenges

Animal breed recognition has practical value for pet identification, veterinary triage, and shelter management. Our work aims to improve real-world robustness by developing a system that classifies known breeds while rejecting non-target images.

Fine-grained breed recognition [18] is challenging due to high inter-class similarity and substantial intra-class variation. Distinguishing features (facial structure, coat texture, color pattern, and body proportion) are often subtle and can be further obscured by variations in pose, illumination, background clutter, and image quality.

1.2. Proposed Approach

Traditional classification methods [15] such as k-nearest neighbours (kNN) [2], classification and regression trees (CART) [8], and support vector machines (SVM) [1] can be applied to animal recognition when suitable handcrafted features are available. However, fine-grained inter-breed classification depends on subtle spatial cues such as facial structure, coat texture, and body shape, making CNNs more suitable because they learn [5] these visual features directly from images.

Based on this motivation, we adopted a comparative and incremental experimental approach. We first established a custom CNN and systematically improved it by testing different architectural, preprocessing, and training configurations. The strongest configuration was then used as a reference for additional CNN- and transformer-based architectures, allowing us to compare models trained from scratch with models using transfer learning.

2. Related Work

Pet recognition has commonly been studied [6] using benchmarks such as the Oxford-IIIT Pet Dataset [12], which contains multiple cat and dog breeds with substantial variation in pose, scale, and appearance. CNN-based methods have been applied to this dataset and similar breed-recognition tasks [6, 10, 16].

CNN architectures have shown strong performance in image recognition. ResNet [3] uses residual connections to support deeper networks, while EfficientNet [17] improves efficiency through balanced scaling of depth, width, and input resolution. Transformer-based models such as Swin Transformer [7] additionally capture both local and global visual relationships through hierarchical attention.

3. Methods

3.1. Dataset

A merged pool of Oxford-IIIT Pet [12] and Stanford Dogs [4] images, 5432 images across 21 classes (10 cat breeds, 10 dog breeds, 1 reject class), 175 ± 25 samples per cat and dog classes, and 1600 reject images was used.

3.1.1 Train/Validation Split and Held-Out Test Set

All architecture and hyperparameter decisions were made using a stratified 80/20 train/validation split drawn from the merged data pool. This ratio provides a practical balance: the 80% training subset gives the models sufficient data to learn meaningful visual patterns, while the 20% validation subset remains large enough to provide stable and representative performance estimates. Stratification was used to preserve class proportions in both subsets, prevent underrepresented classes from being omitted, and provide more reliable and representative validation results.

Data augmentation was applied only to the 80% training subset, using `torchvision.transforms`, whereas the validation subset used deterministic resizing and normalization. Although this split measures how well a model generalizes to unseen samples from the known data distribution, it does not fully evaluate generalization beyond that distribution. Therefore, the 143-image test set was kept completely isolated from all model selection, hyperparameter tuning, and ensembling steps.

3.1.2 Augmentation

To improve generalization, augmentation was applied exclusively to the training images. At each training epoch, a newly sampled stochastic augmentation was applied to each image. Random resized crops of size 224×224 were used with a scale range of 0.75–1.0 and an aspect-ratio range of 0.75–1.33. Horizontal flipping was applied with probability $p = 0.5$, together with random rotations of $\pm 10^{\circ}$ and color jitter with brightness and contrast values of 0.15, saturation of 0.10, and hue of 0. Grayscale conversion, perspective transformation, and random erasing were disabled. All images were converted to tensors and normalized. During validation and inference, images were resized to 256 pixels, followed by a 224×224 center crop and the same normalization procedure.

3.2. Strategy

We followed an incremental optimization strategy. Starting from an initial baseline configuration, one variable was changed at a time to isolate its effect on validation performance. When a modification produced an improvement, the resulting configuration was used as the new baseline for the next experiment. This process was repeated for architecture, preprocessing, regularization, training duration, and confidence-based rejection settings.

3.3. Design Choices

The same initial settings were used across models for fair comparison. Architecture-specific values were changed only when experiments showed different behavior.

YOLO Localization: The largest detected cat or dog bounding box was selected using YOLOv8n, with a confidence threshold of 0.25 and 10% padding around the detected animal region. The resulting region was converted into a square crop before classification. If no valid animal was detected, the original image was forwarded to the classifier instead of being assigned directly to the reject class, since removing this fallback caused classification errors.

Optimizer and Loss Function: AdamW was used for effective regularization, while cross-entropy loss was applied for multi-class classification.

Label Smoothing: To reduce overfit, it was set to 0.1.

Learning Rate and Weight Decay: The initial maximum learning rate was set to 10^{-3} for the from-scratch models and 10^{-4} for the pretrained models. The lower rate was used during fine-tuning to avoid disrupting the useful features already learned by the pretrained weights. The initial weight decay was set to 10^{-4} for all models.

Learning Rate Scheduler: Linear warm-up followed by cosine decay [9] was used for stable optimization.

Epochs: Training started with 30 epochs. If validation performance was still improving, training was extended to 50, 80, or 100 epochs. Early stopping was used when the validation macro-F1 score did not improve within the configured patience period.

Checkpoint Selection: The checkpoint with the best validation result was retained instead of the checkpoint from the final epoch.

Input Size: The models used 224×224 images. This resolution preserved fine-grained breed details while remaining compatible with the pretrained architectures.

Confidence Thresholding: For the final models, the confidence threshold was optimized using the checkpoint with the highest validation accuracy. The threshold was included only when it produced a measurable improvement.

3.4. From Scratch Models

3.4.1 Custom CNN

In the preliminary report, the from-scratch CNN selected as the baseline model was systematically evaluated under a wide range of configurations, with each experiment designed to iteratively improve its performance and refine the final architecture.

To determine an efficient network depth, architectures with 4, 8, and 12 convolutional layers were compared.

The 4-layer model was too shallow, achieving an accuracy of 48.29%, whereas the 8-layer and 12-layer models achieved comparable accuracies of 61.20% and 59.45%, respectively, indicating performance saturation beyond eight layers. Therefore, the custom CNN was designed with eight convolutional layers and approximately 1.18 million parameters. These layers were organized into four feature-extraction blocks, each containing two 3×3 convolutional layers followed by batch normalization and ReLU activation, with a 2×2 max-pooling operation applied at the end of each block. After establishing the optimal network depth, the kernel size was further evaluated by comparing 3×3 and 5×5 kernels. Since the 5×5 kernels resulted in lower accuracy and required more parameters, the 8-layer CNN with 3×3 kernels was retained as the final architecture.

Following architectural optimization, augmentation strength was systematically evaluated on the 8-layer CNN. The medium configuration, consisting of a crop scale of 0.65–1.0, a rotation of $15°$, color jitter with brightness and contrast values of 0.25, saturation of 0.20, and hue of 0.03 applied with a probability of 0.7, together with grayscale, perspective transformation, and random erasing applied with probabilities between 0.05 and 0.10, achieved the highest accuracy of 61.66%. A stronger augmentation setting was also tested, but it reduced performance, suggesting that excessive image perturbations made the already challenging fine-grained classification task more difficult rather than improving generalization. Medium augmentation with these hyperparameters was therefore selected for the remaining experiments.

To control overfitting, dropout rates of 0.20, 0.25, and 0.30 were evaluated, with 0.30 yielding the highest accuracy. The strength of label smoothing [11], which acts as a regularization method by preventing overly confident predictions, was examined by reducing it from 0.10 to 0.05. However, this decreased accuracy from 61.66% to 59.20%, and 0.10 was therefore retained. Also, decreasing the learning rate from 0.001 to 0.0007 resulted in lower accuracy, leading to the retention of 0.001 in the final configuration.

A wider CNN configuration with channel dimensions of 48, 96, 192, and 384 was also evaluated. Although this architecture contained approximately 2.65 million parameters, more than twice the parameter count of the standard 8-layer CNN, it did not provide superior performance. The standard model achieved higher accuracy, macro-F1, and reject-F1 scores while using fewer parameters and was therefore selected as the more computationally efficient configuration.

Incorporating YOLO-based preprocessing increased the validation accuracy of the 8-layer CNN from 61.66% to 67.64%. The number of training epochs was subsequently evaluated using 30, 50, 80, and 100 epochs. Performance improved progressively with longer training, with 100 epochs achieving the highest validation accuracy of 75.48%. Therefore, 100 epochs were selected for the final configuration. Based on this result, the other from-scratch models were also trained for 100 epochs.

All architectural and training ablations were compared using raw validation predictions without confidence thresholding. After model selection, the confidence threshold was calibrated on the fixed validation split as post-processing. The final configuration achieved a validation accuracy of 77.97%.

3.4.2 ResNet18 From-Scratch

ResNet18 was evaluated as the residual-network model in the from-scratch experiments, with approximately 11.2 million trainable parameters. The initial validation accuracy was 66.54%. When YOLO cropping was applied only during validation, performance decreased substantially to 57.42%. This showed that using different preprocessing procedures during training and validation caused a distribution mismatch. For this reason, later experiments applied the same preprocessing during both training and validation. After adding YOLO preprocessing, validation accuracy increased to 69.03% without confidence thresholding.

For this model, detector confidence thresholds of 0.20, 0.25, and 0.30 were evaluated together with padding ratios of 7.5%, 10%, 12.5%, and 15%. The combination of a 0.25 confidence threshold and 10% padding achieved the best validation performance. Therefore, this configuration was selected as the common YOLO preprocessing setup for all models. Also, dropout values of 0.0 and 0.1 were evaluated. Although dropout slightly reduced false accepts, the configuration without dropout provided a better balance across the remaining metrics and was therefore retained.

The best ResNet18 checkpoint was obtained in the increased 100-epoch run at epoch 85. Before threshold calibration, it achieved 77.14% validation accuracy. The selected threshold was 0.22, increasing the validation accuracy to 77.70%. These results established ResNet18 as a strong from-scratch residual model and a useful component of the from-scratch ensemble.

3.4.3 EfficientNet-B0 From-Scratch

EfficientNet-B0 was trained from scratch to test whether a compact convolutional architecture could perform competitively with ResNet18. The model has approximately 4.0 million trainable parameters, which is substantially fewer than ResNet18, while its MBConv blocks and compound scaling provide a stronger architecture than the custom CNN baseline. The same preprocessing steps used for the other models were applied.

The 100-epoch run reached its best checkpoint at epoch 94. After confidence-threshold calibration, a threshold of

0.52 was selected, producing 74.75% validation accuracy, a macro-F1 score of 76.31%, and a reject-class F1 score of 69.91%. Despite weaker performance than the other from-scratch models, it provided complementary ensemble predictions.

3.4.4 Other Models

Swin Transformer Tiny: Training was stopped after epoch 10 because the validation accuracy did not improve within the configured patience period. The best validation accuracy was 11.52%, and disabling early stopping did not lead to any further improvement; instead, validation accuracy decreased during continued training. Additional experiments with different dropout, attention-dropout, and weight-decay values also failed to improve performance. With approximately 27.5 million parameters, the model was difficult to optimize from random initialization using the limited training data, suggesting poor convergence rather than effective representation learning.

ResNet34 and ResNet50: ResNet architectures are commonly named according to their depth, such as ResNet18, ResNet34, and ResNet50, where the suffix indicates the number of layers in the architecture. [13] In our from-scratch training experiments, increasing the network depth did not improve performance. When ResNet34 was trained using the optimized ResNet18 configuration, it achieved approximately 60% validation accuracy, which was lower than that of ResNet18. A similar decrease was also observed with ResNet50. Therefore, ResNet18 was selected as the final architecture from this family because it provided a better balance between optimization stability, and generalization.

3.5. Pretrained Models

3.5.1 Pretrained ResNet18

This model contains approximately 11.2 million trainable parameters. Training was initially planned for 30 epochs, but validation performance was still improving, so early stopping was disabled and training was extended to 50 epochs. Progress became limited near the end of training, and the best checkpoint was obtained before the final epoch.

The highest validation accuracy was achieved at epoch 47, reaching 93.55% accuracy. With the calibrated confidence threshold of 0.16, validation accuracy increased to 93.64% and macro-F1 to 94.23%, clearly outperforming the from-scratch ResNet18 configuration.

3.5.2 Pretrained EfficientNet-B0

The pretrained EfficientNet-B0 model was fine-tuned as a compact transfer-learning architecture with approximately 4.0 million trainable parameters. Compared with the larger pretrained ResNet18 and Swin-Tiny models, it provided a smaller architecture while still benefiting from ImageNet-pretrained feature representations.

The best checkpoint was selected at epoch 30. Without confidence thresholding, EfficientNet-B0 achieved 93.82% validation accuracy and a macro-F1 score of 93.87% on the fixed validation split. Confidence thresholding further improved performance. With the selected threshold of 0.37, validation accuracy increased to 94.10% and macro-F1 increased to 94.29%. The reject-class behavior also became more conservative, reducing false accepts from 34 to 29, while false rejects increased from 6 to 12.

3.5.3 Pretrained Swin-Transformer-Tiny

The model was initially fine-tuned with all 27.5 million parameters trainable, achieving approximately 89% validation accuracy. Since the model capacity was high relative to the limited dataset size, alternative fine-tuning strategies were also evaluated. Training only the classification head resulted in a validation accuracy of 87.37%, whereas unfreezing only the final stage achieved 93.73% with approximately 14.2 million trainable parameters. However, full fine-tuning was retained in the final implementation to maintain a consistent and reproducible training pipeline across the evaluated architectures.

Increasing weight decay slightly reduced performance, so the original value was retained. Same preprocessing steps were included to keep the pipeline consistent with the other models, although it was a negligible improvement. Confidence thresholding produced only a negligible improvement in validation accuracy. Using a threshold of 0.39 on the checkpoint from epoch 29, accuracy increased from 94.47% to the model's final accuracy of 94.75%.

3.6. Ensemble Strategy

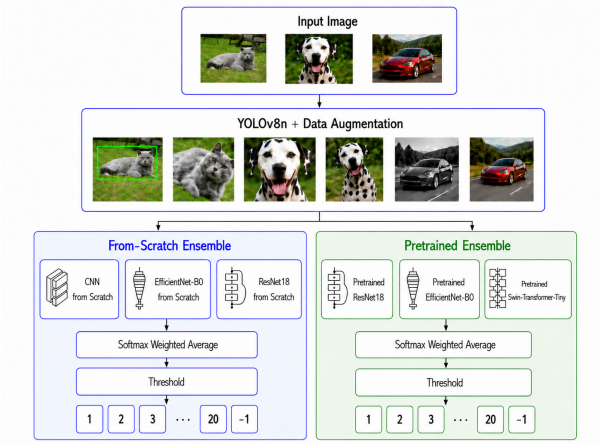

Figure 1. Overview of the proposed pipeline.

As illustrated in Figure 1, the selected models were divided into from-scratch and pretrained groups, and a separate en-

semble was created for each. This allowed us to compare the combined performance of models learned from random initialization with models using transferred representations.

We used a weighted average of the models' softmax probabilities. The ensemble weights were determined through a grid search over combinations with increments of 0.05, using the predictions from each model's best validation checkpoint. The combination yielding the highest validation accuracy was selected for each ensemble. For three models, the final probability vector was computed as:

$$\mathbf{P}_{\text{final}} = w_1\mathbf{P}_1 + w_2\mathbf{P}_2 + w_3\mathbf{P}_3, \tag{1}$$

$$w_1 + w_2 + w_3 = 1. \tag{2}$$

The final class prediction was then obtained as

$$\hat{y} = \arg\max_c \left(P_{\text{final},c}\right), \tag{3}$$

where $P_{\text{final},c}$ denotes the ensemble probability assigned to class c. This approach aimed to reduce model-specific errors and improve prediction stability by assigning greater influence to the stronger models. The weights used for both ensembles are given below.

From-Scratch Ensemble Weights: The custom CNN, ResNet18, and EfficientNet-B0 were assigned weights of 0.45, 0.25, and 0.30, respectively. Therefore, the probability of the from-scratch ensemble was calculated as

$$\mathbf{P}_{\text{scratch}} = 0.45\mathbf{P}_{\text{CustomCNN}} + 0.25\mathbf{P}_{\text{ResNet18}} + 0.30\mathbf{P}_{\text{EfficientNetB0}} \tag{4}$$

Pretrained Ensemble Weights: Swin Transformer Tiny, ResNet18, and EfficientNet-B0 were assigned weights of 0.30, 0.35, and 0.35, respectively. Therefore, the pretrained ensemble probability was calculated as

$$\mathbf{P}_{\text{pretrained}} = 0.30\mathbf{P}_{\text{SwinTiny}} + 0.35\mathbf{P}_{\text{ResNet18}} + 0.35\mathbf{P}_{\text{EfficientNetB0}} \tag{5}$$

4. Experiments

4.1. Evaluation and Experimental Analysis

4.1.1 Training and Validation Curves

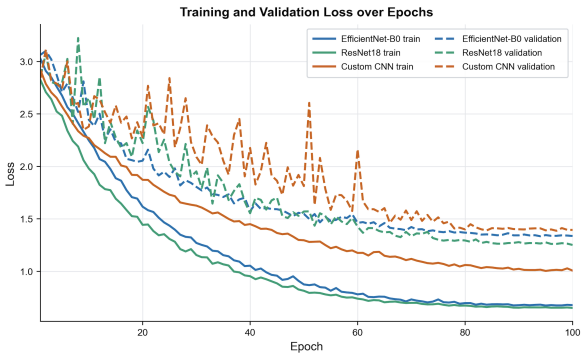

Figure 2. Training and validation loss curves of the three from-scratch models. Solid lines represent training loss, while dashed lines represent validation loss.

As shown in Figure 2, the training and validation losses decreased across all from-scratch models and largely stabilized after approximately 70–80 epochs. ResNet18 achieved the lowest final validation loss, whereas the Custom CNN exhibited the smallest train–validation gap but higher overall loss values. This indicates that the Custom CNN showed less evidence of overfitting, although its overall predictive performance remained weaker.

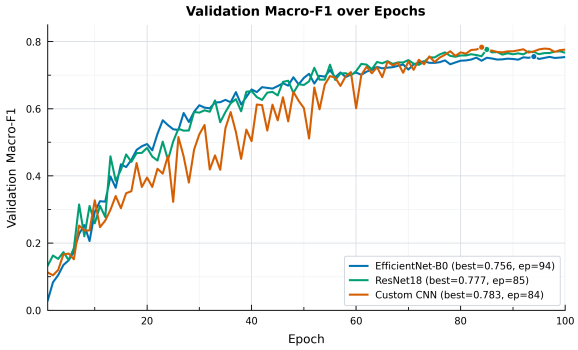

Figure 3. Validation macro-F1 scores of the three from-scratch models over 100 epochs.

As demonstrated in Figure 3, the Custom CNN shows strong fluctuations in the early epochs but ultimately outperforms the larger from scratch architectures. Its lower parameter count may have provided a more suitable capacity for the limited dataset, reducing the risk of overfitting while preserving sufficient representational power. EfficientNet-B0 followed a smoother but lower-performing trajectory, indicating that its compound-scaled design offered limited benefit without pretrained initialization under the adopted experimental configuration. The convergence of all three models to a similar plateau after approximately 70 epochs indicates that further training alone was unlikely to produce substantial gains under the selected setup.

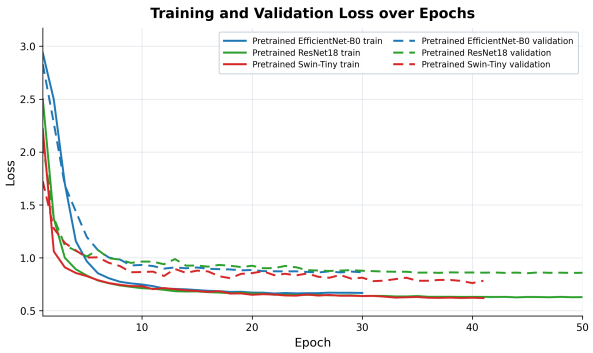

Figure 4. Training and validation loss curves of the three pretrained models. Solid lines represent training loss, dashed lines represent validation loss.

As shown in Figure 4, ResNet18 and EfficientNet-B0

reached comparable training losses but diverged in validation loss, with ResNet18 plateauing higher despite longer training. This suggests weaker generalization under the selected configuration rather than insufficient training time. Swin-Tiny reached the lowest validation loss earliest, indicating that its attention-based representations transferred more effectively to unseen breed variations. Although train–validation gaps remained visible, they did not widen substantially over time, indicating that the applied regularization limited severe overfitting across all three models.

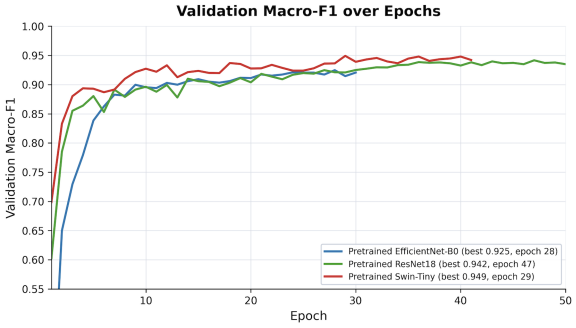

Figure 5. Validation macro-F1 scores of the three pretrained models across training epochs.

As given in Figure 5, transfer learning enabled all three models to reach strong performance within few epochs. Swin-Tiny achieved the highest macro-F1, suggesting that its hierarchical attention captured the subtle visual differences between breeds most effectively. EfficientNet-B0 plateaued earlier, indicating a lower performance ceiling, whereas ResNet18 improved more slowly. Overall, the results show that Swin-Tiny converged to the best validation performance, whereas the other models offered slightly lower but more stable learning trajectories.

4.1.2 Quantitative Performance Comparison

From-Scratch Model	Accuracy	Macro-F1	Macro-Precision	Macro-Recall	Reject-F1	False Accepts	False Rejects
Custom CNN	0.7797	0.7975	0.8057	0.7986	0.7270	87	88
EfficientNet-B0	0.7475	0.7631	0.7970	0.7419	0.6991	76	134
ResNet18	0.7770	0.7827	0.7851	0.7854	0.7574	78	77
From-Scratch Ensemble	**0.8230**	**0.8384**	**0.8414**	**0.8400**	**0.7738**	**72**	**73**

Table 1. Performance of the from-scratch models and ensemble.

Pretrained Model	Accuracy	Macro-F1	Macro-Precision	Macro-Recall	Reject-F1	False Accepts	False Rejects
EfficientNet-B0	0.9410	0.9429	0.9349	0.9526	0.9342	29	12
ResNet18	0.9364	0.9423	0.9387	0.9476	0.9175	31	21
Swin-Tiny	0.9475	0.9527	0.9392	0.9686	0.9296	36	**7**
Pretrained Ensemble	**0.9548**	**0.9595**	**0.9506**	**0.9695**	**0.9404**	**28**	9

Table 2. Performance of the pretrained models and ensemble.

The values in Tables 1 and 2 provide a more detailed view of each model's behavior beyond overall accuracy. Accuracy was used as the primary selection metric since it reflects the proportion of correctly classified samples, while macro-F1 was considered next to evaluate whether performance remained balanced across all classes.

$$F1_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}, \tag{6}$$

$$\text{Macro-F1} = \frac{1}{21}\sum_{c=1}^{21} F1_c. \tag{7}$$

where TP_c, FP_c, and FN_c denote the true positives, false positives, and false negatives for class c, respectively. Macro-F1 was calculated by averaging the F1-scores of all 21 classes.

Macro-precision and macro-recall were used to examine false positive and false negative tendencies, and reject-class F1 measured how reliably non-target samples were handled. The false-accept and false-reject counts further showed the trade-off between accepting unknown samples as target breeds and rejecting valid target images.

The pretrained models consistently achieved stronger results than the from-scratch models. In both groups, the ensemble obtained the highest accuracy and macro-F1, indicating that combining complementary models improved both overall correctness and class-level balance. The pretrained ensemble achieved the best overall performance, while the from-scratch EfficientNet-B0 showed the weakest accuracy and reject-class performance.

4.1.3 Confusion Matrix

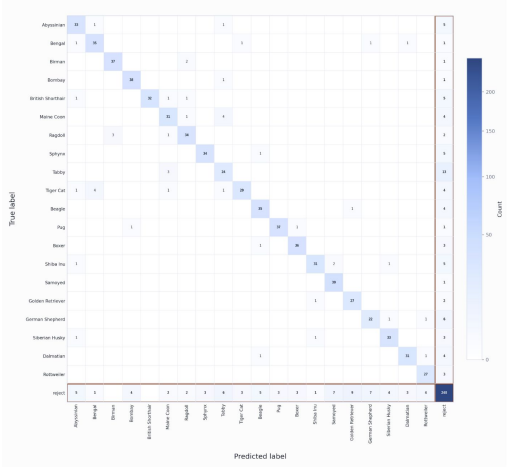

Figure 6. Confusion matrix of the from-scratch ensemble on the validation set. The last row and column represent the reject class.

As shown in Figure 6, the dominant source of error was not confusion between cats and dogs, but fine-grained discrimination within the same animal group. Errors were particularly concentrated among visually similar cat breeds, indicating that the from-scratch ensemble learned broad animal-level features more reliably than subtle breed-specific characteristics. Moreover, false accepts were distributed across

several breed classes rather than being dominated by a single class, suggesting general open-set ambiguity instead of a systematic class-specific bias. Overall, the model separated the major animal groups effectively, but its main limitations were fine-grained feline discrimination and confidence calibration near the reject boundary.

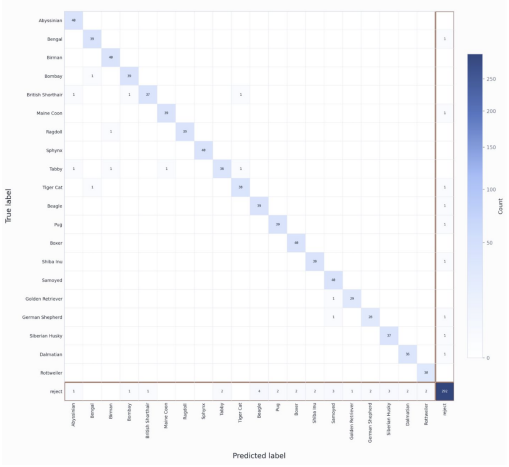

Figure 7. Confusion matrix of the pretrained ensemble on the validation set. The last row and column correspond to the reject class.

As demonstrated in Figure 7, the pretrained ensemble achieved substantially stronger class separation than the from-scratch ensemble, with only sparse off-diagonal errors. Most breeds were classified almost perfectly, indicating that transfer learning provided more discriminative fine-grained representations. Tabby remained the most challenging feline class, with its errors distributed across several visually similar cat breeds rather than concentrated on a single class, suggesting broad visual overlap. Dog-breed confusion was minimal, showing that the learned representations captured their distinguishing features more reliably. The reject mechanism also improved markedly, correctly rejecting 292 samples with only 28 false accepts and 9 false rejects; indicating a slightly permissive decision boundary that favors retaining valid target images over rejecting every unknown input. Overall, the matrix demonstrates strong breed-level discrimination and effective, though not fully conservative, rejection performance.

4.2. Debugging with xAI

Grad-CAM [14] was used to inspect whether the models focused on the animal regions or on irrelevant background cues. Since the final predictions were produced by ensembles, Grad-CAM was applied to representative high-performing individual models from each ensemble family: the Custom CNN from the from-scratch group and Swin-Tiny from the pretrained group. This made the explanations easier to interpret because Grad-CAM is model-specific and does not directly visualize the averaged ensemble output.

The xAI analysis also supported the decision to retain the raw-image fallback. During Grad-CAM generation, YOLO-crop preprocessing was used whenever a valid cat or dog detection was available. However, when YOLO failed to produce a valid crop, the affected sample was not discarded. Instead, the visualization was generated from the original image using the fallback preprocessing. This showed that directly rejecting images after detector failure could remove valid target-breed samples before classification. Therefore, the final pipeline retained the raw-image fallback, allowing the classifier to still make a prediction while the calibrated confidence threshold controlled the final reject decision. This made the system more robust than a strict detector-gated approach.

Figure 8. Grad-CAM visualizations for representative cat and dog samples using the Custom CNN and Swin-Tiny models.

In addition, the Grad-CAM analysis revealed a minor limitation of the pretrained Swin-Tiny model. As shown in Figure 8, the Custom CNN focused more consistently on the target animal, while Swin-Tiny occasionally attended to less relevant regions. Despite its high validation accuracy of 94.75%, some visualizations suggested that the model occasionally focused on less relevant image regions rather than the target animal (above given Swin Tiny cat example), indicating possible reliance on dataset-specific visual cues. In contrast, the from-scratch Custom CNN focused more consistently on the target animal. These isolated cases may indicate mild overfitting due to the model's high capacity relative to the limited dataset size.

5. Conclusions

5.1. Test Set Application

The two ensemble models were evaluated on the held-out test set to determine both their generalization performance and computational suitability. Ensembling was preferred to reduce model-specific errors, limit the effect of overfitting, and produce more stable predictions by averaging the outputs of complementary architectures. A model was considered computationally suitable if every test image was processed in less than five seconds.

From-Scratch Ensemble: The from-scratch ensemble was evaluated on the provided 143 -image test set using a confidence threshold of 0.32 . It achieved an accuracy of 68.53% and a macro-F1 score of 63.96%. Its average inference time was 0.134 seconds per image, with a maximum processing

time of 3.76 seconds. It also met the timing requirement, but performed worse than the pretrained ensemble.

Pretrained Ensemble: The pretrained ensemble was evaluated on the same held-out test set using a confidence threshold of 0.30. It achieved an accuracy of 79.02% and a macro-F1 score of 76.95%. Its average inference time was 0.159 seconds per image, while the maximum time was 3.56 seconds. Therefore, it satisfied the five-second requirement.

Since both ensemble configurations satisfied the predefined computational constraint, the comparison was limited to the ensembles themselves rather than their individual constituent models.

5.2. Final Model

Among the two ensemble configurations, the pretrained ensemble was selected as the final model because it provided the strongest and most consistent performance on both the validation and held-out test sets. It achieved a validation accuracy of 95.48% and a macro-F1 score of 95.95%, while also outperforming the from-scratch ensemble. These results indicate that the pretrained ensemble generalized more effectively and was less vulnerable to model-specific errors and overfitting. Combining ResNet18, EfficientNet-B0, and Swin Transformer Tiny also had a positive effect, since the models captured complementary visual representations and their weighted predictions reduced individual classification errors. Based on this balance of accuracy, class-level performance, rejection reliability, and generalization, the pretrained ensemble was selected as the final model.

Class	F1	Precision	Recall
Abyssinian	0.9639	0.9302	1.0000
Bengal	0.9630	0.9512	0.9750
Birman	0.9756	0.9524	1.0000
Bombay	0.9630	0.9512	0.9750
British Shorthair	0.9487	0.9737	0.9250
Maine Coon	0.9750	0.9750	0.9750
Ragdoll	0.9873	1.0000	0.9750
Sphynx	1.0000	1.0000	1.0000
Tabby	0.9231	0.9474	0.9000
Tiger Cat	0.9500	0.9500	0.9500
Beagle	0.9398	0.9070	0.9750
Pug	0.9630	0.9512	0.9750
Boxer	0.9756	0.9524	1.0000
Shiba Inu	0.9630	0.9512	0.9750
Samoyed	0.9412	0.8889	1.0000
Golden Retriever	0.9667	0.9667	0.9667
German Shepherd	0.9333	0.9333	0.9333
Siberian Husky	0.9487	0.9250	0.9737
Dalmatian	0.9600	0.9474	0.9730
Rottweiler	0.9677	0.9375	1.0000
Reject	0.9404	0.9701	0.9125

Table 3. Per-class metrics of the pretrained 50-epoch ensemble. All rate-based metrics are reported on a scale from 0 to 1.

As shown in Table 3, the pretrained ensemble achieved consistently strong performance across all classes, with per-class F1 scores ranging from 0.9231 to 1.0000. Sphynx obtained perfect precision, recall, and F1, which may be explained by its highly distinctive visual characteristics, particularly its hairless appearance and prominent facial structure. In contrast, Tabby and German Shepherd produced the lowest F1 scores, suggesting greater visual overlap with similar breeds. Samoyed achieved perfect recall but lower precision, indicating that some samples from other classes were incorrectly predicted as Samoyed. The reject class obtained an F1 score of 0.9404, with higher precision than recall, showing that rejected predictions were generally reliable, although some non-target images were still accepted as known breeds. Overall, despite the limited number of approximately 35 ± 5 validation samples per breed, the narrow F1 range across the target classes and the strong reject-class performance indicate that the pretrained ensemble generalized consistently without sacrificing rejection reliability.

5.3. Conclusion

This study investigated the fine-grained classification of 20 cat and dog breeds together with an explicit reject class. Several CNN- and transformer-based architectures were evaluated under both from-scratch and transfer-learning settings using an incremental optimization strategy. Consistent YOLO-based localization, data augmentation, confidence-threshold calibration, and weighted softmax ensembling improved both breed classification and rejection performance.

Pretrained and fine-tuned architectures consistently outperformed their from-scratch counterparts, demonstrating the value of ImageNet-derived representations for this data-limited task. Weighted softmax ensembling further improved performance in both model groups by reducing architecture-specific errors.

Overall, the results demonstrate that combining complementary pretrained architectures provides a more accurate, balanced, and reliable solution than relying on a single model. The pretrained ensemble achieved the strongest overall results, reaching 95.48% validation accuracy and a macro-F1 score of 95.95%. On the held-out test set, it achieved 79.02% accuracy and a macro-F1 score of 76.95%, while satisfying the required inference-time constraint. Based on this combination of accuracy, rejection reliability, and inference efficiency, the pretrained ensemble was selected as the final model.

Future work could focus on expanding the dataset, reducing overfitting, improving confidence under distribution shifts, and reducing confusion between visually similar breeds.

References

[1] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. 1

[2] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967. 1

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 1

[4] Aditya Khosla, Nityananda Jayadevaprakash, Bangpeng Yao, and Li Fei-Fei. Novel dataset for fine-grained image categorization: Stanford dogs. In *First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition*, 2011. 2

[5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, pages 1097–1105, 2012. 1

[6] Zhonglan Lin, Haiying Xia, Yan Liu, Yunbai Qin, and Cong Wang. A method for enhancing the accuracy of pet breeds identification model in complex environments. *Applied Sciences*, 14(16):6914, 2024. 1

[7] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021. 1

[8] Wei-Yin Loh. Classification and regression trees. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 1(1):14–23, 2011. 1

[9] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. 2

[10] S. Divya Meena and L. Agilandeeswari. An efficient framework for animal breeds classification using semi-supervised learning and multi-part convolutional neural network (mp-cnn). *IEEE Access*, 7:151783–151802, 2019. 1

[11] Rafael Müller, Simon Kornblith, and Geoffrey E. Hinton. When does label smoothing help? In *Advances in Neural Information Processing Systems*, 2019. 3

[12] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, and C. V. Jawahar. Cats and dogs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3498–3505, 2012. 1, 2

[13] PyTorch. Torchvision transforms. https://docs.pytorch.org/vision/stable/transforms.html, 2023. Accessed: 2026-07-01. 4

[14] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 618–626, 2017. 7

[15] Komal Shah and Dr. Patel. Study of multiclass classification techniques. *IARJSET*, 11, 2024. 1

[16] S. Sudharson, S. Deena, and Sappidi Nischinth Reddy. Efficient real-time breed classification using yolov7 object detection algorithm. In *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6, 2023. 1

[17] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6105–6114, 2019. 1

[18] Xiu-Shen Wei, Yi-Zhe Song, Oisin Mac Aodha, Jianxin Wu, Yuxin Peng, Jinhui Tang, Jian Yang, and Serge Belongie. Fine-grained image analysis with deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(12):8927–8948, 2022. 1

Appendix

The source code and implementation details are available at the GitHub link:

https://github.com/OrhunMahir/SEP_Project

Author Contributions

Doga Budakci

- Preliminary Report:
  - Abstract
  - 4.1.1 Baseline CNN Proposed Architecture
  - 4.1.2 Training Strategy / Design Choices
  - 4.2.1 Proposed Strategies / Design Choices
  - 5. Conclusion and Future Work
- Code:
  - Design of the custom CNN architecture from scratch
  - Development and Training of the custom CNN from scratch
  - Development and Training of the pretrained EfficientNet-B0 model
  - Development and Training of the pretrained ResNet18 model
  - Development and Training of the ResNet50 model from scratch
  - Train and Validation Curve & Analysis
- Final report:
  - 3.1.2 Augmentation
  - 3.4.1 Custom CNN
  - 3.3 Design Choices
  - 3.6 Ensemble Strategy
  - 4.1.1 Training and Validation Curves
  - 4.1.2 Quantitative Performance Comparison
  - 5.2 Final Model
  - 5.3 Conclusion
  - Design and Structure of Report

Doga Erdag

- Preliminary Report:
  - Abstract
  - 4.1.2 Training Strategy / Design Choices
  - 4.2. Candidate Architectures
  - 4.2.1 Proposed Strategies / Design Choices
  - 4.3. Evaluation Plan
- Code:
  - First draft of custom CNN
  - Development and Training of the Swin-Tiny model from scratch
  - Development and Training of the ResNet34 model from scratch
  - Development and Training of the ResNet50 model from scratch
  - Development and Training of pretrained Swin-Tiny model
  - First draft of pretrained ResNet18 model
- Final report:
  - 3.1.1 Train/Validation Split and Held-Out Test Set
  - 3.1.2 Augmentation
  - 3.2 Strategy
  - 3.4.4 Other Models
  - 3.5.3 Pretrained Swin-Transformer-Tiny
  - 3.6 Ensemble Strategy
  - 5.2 Final Model
  - 5.3 Conclusion
  - Design and Structure of Report

Eren Degirmenci

- Preliminary Report:
  - 1. Introduction
  - 1.1. Motivation
  - 1.2. Challenges
  - 2.1. Different CNN Architectures
  - Literature Research
  - Design and Structure of Report
- Code:
  - Development and Training of the EfficientNet-B0 model from scratch
  - Grad-CAM Analysis code
  - Grad-CAM Implementation of Swin-Tiny Model
  - Confusion Matrix
  - Ensemble Design and Training of pretrained models
  - Dataset Preparation
- Final report:
  - Abstract
  - 1 Introduction
  - 1.1 Motivation and Challenges
  - 3.1 Dataset
  - 3.5.2 Pretrained EfficientNet-B0
  - 4.1.3 Confusion Matrix
  - 4.2 Debugging with XAI
  - Literature Research

Orhun Mahirogullari

- Preliminary Report:
  - 2. Related Work
  - 2.2. Alternative Approaches
  - 3. Data Research
  - 4.4. XAI
  - Design and Structure of Report
- Code:
  - Establishing the codebase and setting up the repository.
  - Development and Training of from scratch ResNet18 model
  - Grad-CAM Analysis code and Implementation of Custom CNN Model
  - Ensemble Design and training of from scratch models
  - YOLO Crop Experiments
  - Dataset Preparation
- Final report:
  - Abstract
  - 1.2 Proposed Approach
  - 2 Related Work
  - 3.4.2 ResNet18 From-Scratch
  - 3.4.3 EfficientNet-B0 From-Scratch
  - 3.5.1 Pretrained ResNet18
  - 4.2 Debugging with XAI
  - 5.1 Test Set Application
  - Literature Research